



▶ Time Series Forecasting del Player Count nei Giochi Steam

Progetto di Intelligenza Artificiale, a.a. 2025–2026

Beatrice Ravaglia



► Problema affrontato

I videogiochi distribuiti su Steam presentano variazioni significative nel numero di giocatori attivi nel tempo, caratterizzate da dinamiche complesse e influenzate da molteplici fattori, come aggiornamenti del gioco, eventi speciali e promozioni.

Obiettivo: stimare l'evoluzione giornaliera del player count su un orizzonte temporale di 14 giorni.

Approcci esistenti in letteratura

Negli ultimi anni, diversi modelli di Deep Learning sono stati applicati con successo al problema del Time Series Forecasting (TSF).

- ▶ **RNN**: utilizzano connessioni ricorrenti per modellare le dipendenze temporali e catturare l'evoluzione dei pattern nelle sequenze.
- ▶ **LSTM** e **GRU**: varianti delle RNN progettate per gestire efficacemente dipendenze di lungo periodo.
- ▶ **CNN**: architetture che riescono a modellare dipendenze temporali complesse estraendo pattern locali attraverso operazioni di convoluzione.

▶ **TCN**: basate su CNN, sono particolarmente efficaci nell'analisi di sequenze lunghe.

▶ **N-BEATS**: architettura basata su MLP, progettata specificamente per il forecasting di serie temporali e caratterizzata da elevate prestazioni su numerosi benchmark.

▶ **N-BEATSx**: estensione di N-BEATS che integra variabili esogene migliorando la capacità predittiva.

▶ **N-HiTS**: evoluzione di N-BEATS che utilizza una struttura gerarchica multi-risoluzione per catturare pattern a diverse scale temporali, ottenendo risultati state-of-the-art in molti benchmark di forecasting.

Principali librerie utilizzate

NeuralForecast



Optuna



Numpy



Pandas



Scikit-learn



Matplotlib



Costruzione del dataset (1)

Fonte dei dati: **SteamDB** (<https://steamdb.info/>)

Copertura: ~350 giochi Steam

Per ogni gioco sono stati scaricati i file excel contenenti i seguenti dati storici:

- **Player count** giornaliero/orario ▶
"DateTime";"Players";"Average Players"
"2019-07-22 00:00:00";546700;;
"2019-07-23 00:00:00";546245;;
- Numero di **recensioni positive** giornalieri ▶- Numero di **recensioni negative** giornalieri ▶
"DateTime";"Positive reviews";"Negative reviews"
"2015-05-14 00:00:00";395;-38
"2015-05-15 00:00:00";1396;-82
- **Prezzo** giornaliero (€) ▶
"DateTime";"Final price";"Historical Low"
"2012-11-01 17:03:07";7,34;7,34
"2012-11-02 12:56:16";5,49;5,49

Costruzione del dataset (2)

Tramite uno script dedicato, per ciascun gioco sono stati estratti anche i **Markers** in formato JSON. Questi dati identificano eventi speciali spesso associati a picchi positivi o negativi dell'attività dei giocatori.

```
[{  
  "x": 1755786600000,  
  "text": "Silksong release date  
announcement",  
  "id": 49372500,  
  "title": "SD",  
  "date": "2025-08-21 16:30:00"  
}]
```



Data cleaning

Per ciascun gioco, i dati vengono allineati a un **intervallo temporale comune** definito sulla base delle serie dei player count e recensioni. I prezzi e i markers vengono successivamente adattati a tale intervallo (tramite troncamento).

Successivamente vengono applicate le seguenti operazioni di pulizia:

	Gestione missing values	Aggregazione dati	Data type conversion
Player count	Forward fill per i NaN; backfill per NaN iniziali	Da orario a giornaliero (media giornaliera)	Date → datetime Numeri → int
Recensioni	Sostituzione NaN con 0	-	Date → datetime Numeri → int
Prezzi	Inserimento giorni mancanti + Forward fill per i NaN; backfill per NaN iniziali	-	Date → datetime Numeri → float
Markers	-	-	Date → datetime

Feature engineering

- ▶ Per ogni giorno viene calcolata la distanza temporale dall'evento (**Marker**) più vicino. Tale valore viene normalizzato nell'intervallo $[0, 1]$:
 - $1 \rightarrow$ giorno dell'evento
 - valori progressivamente più bassi $\rightarrow$ giorni lontani dall'evento
 - in assenza di markers, l'effetto degli eventi viene posto a 0.
- ▶ Per i **giochi free-to-play**, privi di dati sul prezzo, viene generata una serie temporale con prezzo costante (pari a 0).
- ▶ Calcolo di **feature derivate**
 - Rating Ratio (relazione tra recensioni positive e negative)
 - Numero totale di recensioni
 - Numero totale di recensioni di 7 giorni prima (reviews lag 7)
 - Giorno della settimana
 - Indicatore weekend (is_weekend)
 - Mese dell'anno
 - Player count del giorno precedente (lag 1)
 - Player count di 7 giorni prima (lag 7)
 - Player count di 14 giorni prima (lag 14)

Split del Dataset e Normalizzazione

Split del dataset (~350 giochi):

90%
Training set

10%
Test set

Le feature derivate dalle recensioni e i prezzi vengono normalizzate tramite **Max Scaling**, effettuato separatamente per ciascun gioco.

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

L'effetto degli eventi è già definito nell'intervallo [0, 1].

Non viene invece applicata alcuna normalizzazione al player count né alle feature temporali.

Modelli utilizzati

TCN

Temporal Convolutional Network with dilated causal convolutions

- ▶ Basata su CNN
- ▶ 2016

N-BEATSx

Neural Basis Expansion Analysis with exogenous variables

- ▶ Basata su MPL e N-BEATS
- ▶ 2021

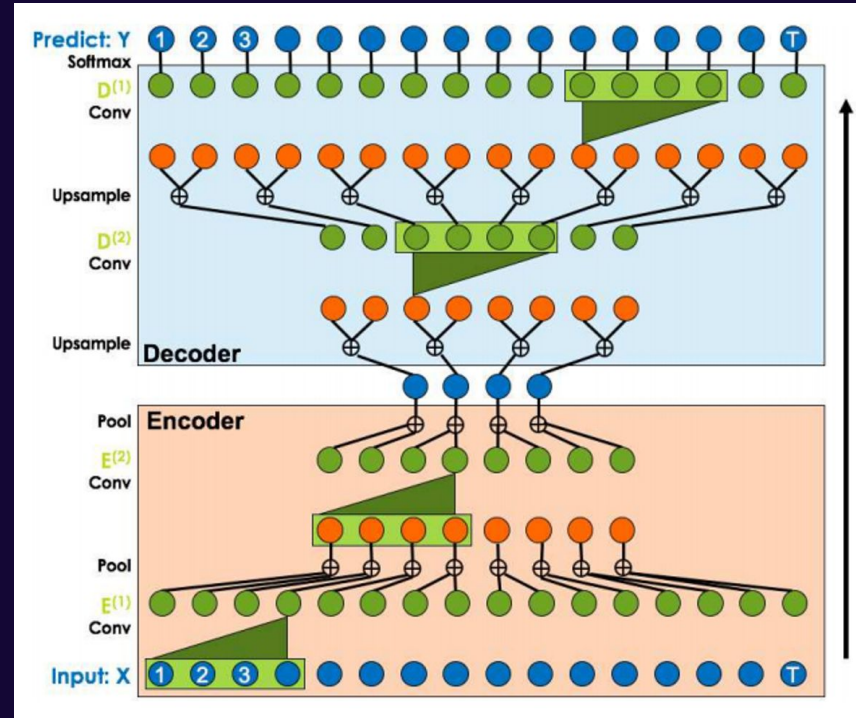
N-HITS

Neural Hierarchical Interpolation for Time Series

- ▶ Basato su MPL, migliora N-BEATS
- ▶ 2022

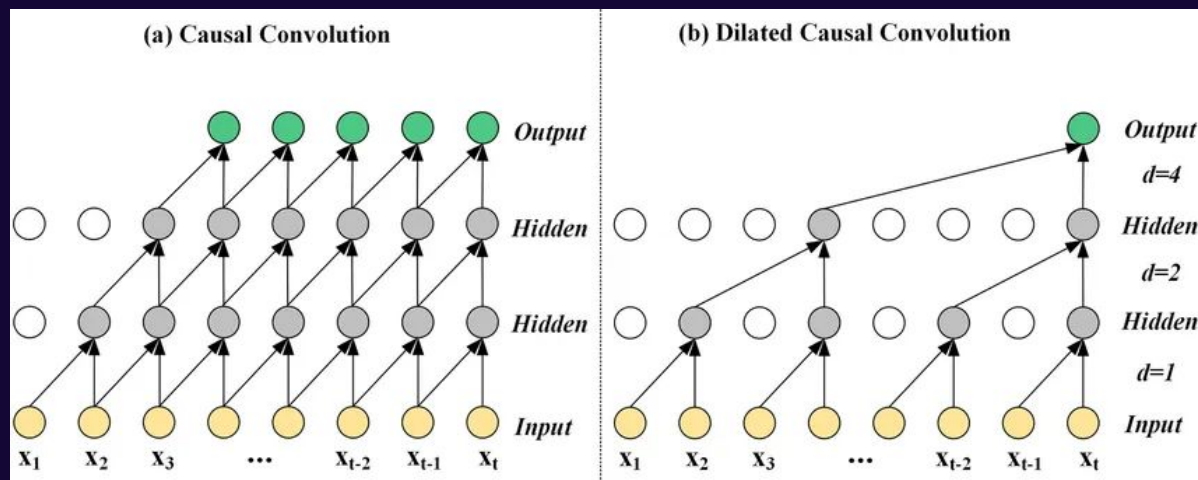
TCN

- ▶ **CNN** (convolutional neural network) adattata per serie temporali
- ▶ l'applicazione del kernel estrae informazioni sulla serie di dati



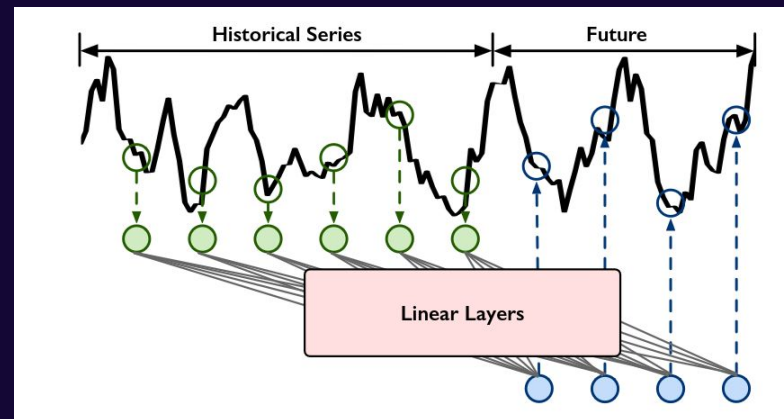
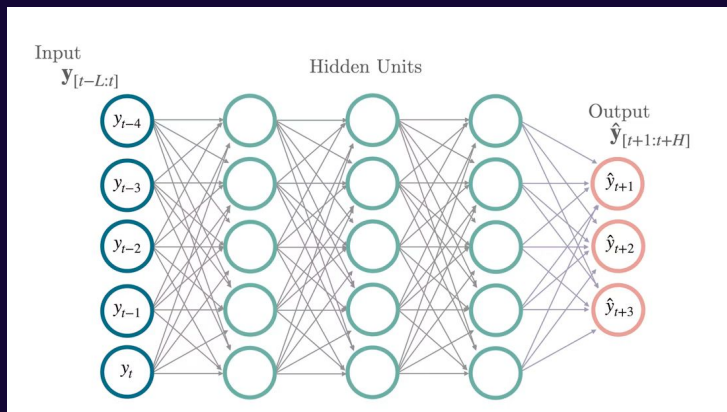
TCN: nel dettaglio

- ▶ causal convolutions: applicare il kernel di convoluzione solo su dati passati
- ▶ per predire utilizzando serie di dati molto lunghe servono molti layer
- ▶ soluzione: dilated causal convolution



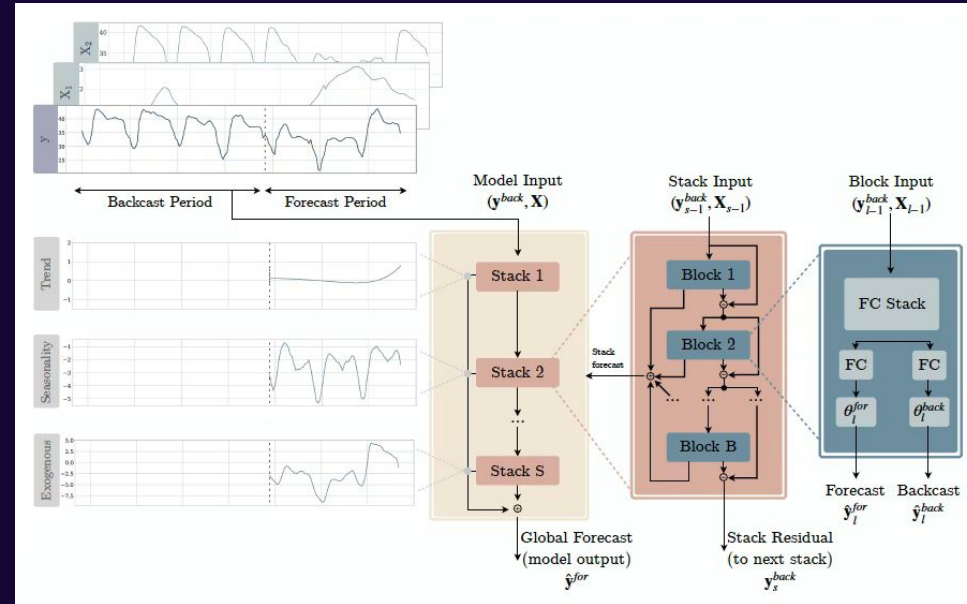
N-BEATSx

- ▶ MLP-based
- ▶ strati di neuroni fortemente connessi



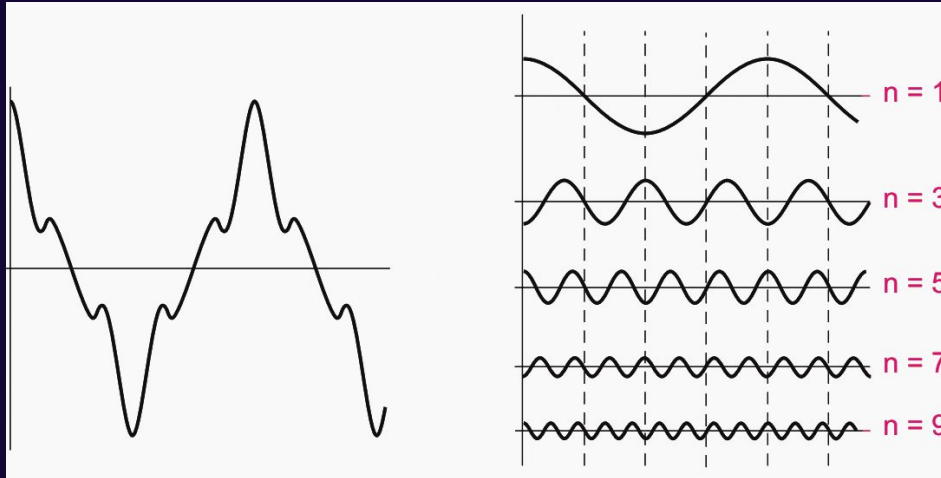
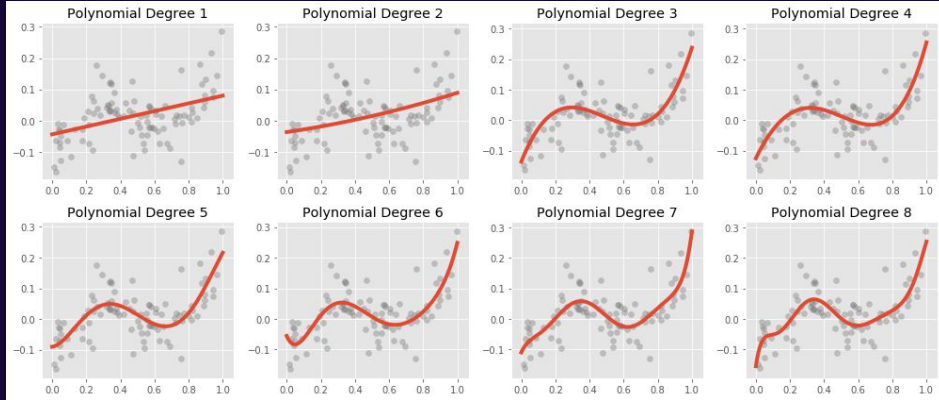
N-BEATSx: nel dettaglio

- ▶ **Basis expansion:** Scomporre la serie temporale proiettando i dati su funzioni matematiche più semplici (funzioni base),
- ▶ Basis expansion create durante il training permettono di riconoscere trend e stagionalità
- ▶ ogni stack cattura un pattern diverso, questi vengono combinati per calcolare la predizione finale



Basis Expansion

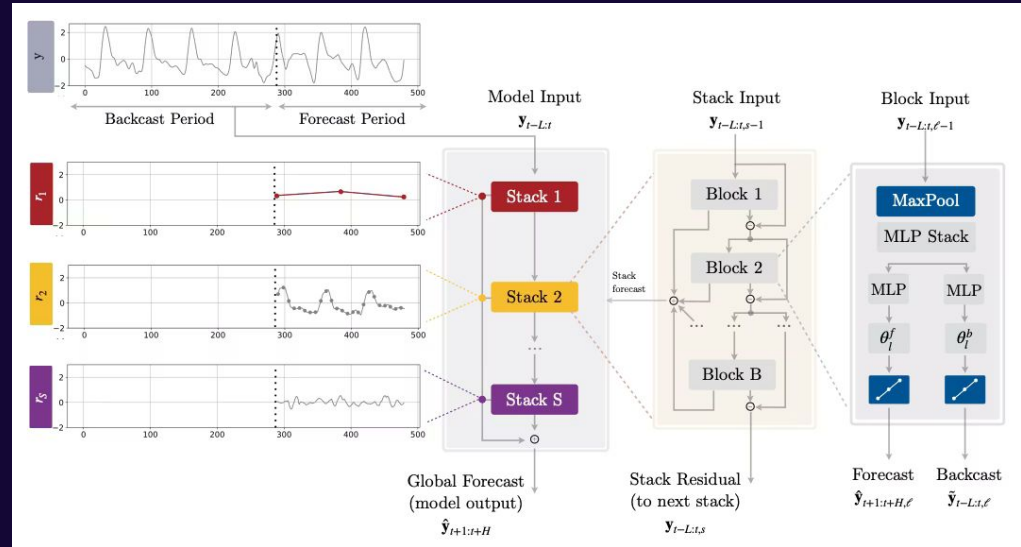
- ▶ **Basi polinomiali** per catturare il trend (andamenti a lungo termine)
- ▶ **Basi armoniche** per la stagionalità



N-HITS

Modifica N-BEATS:

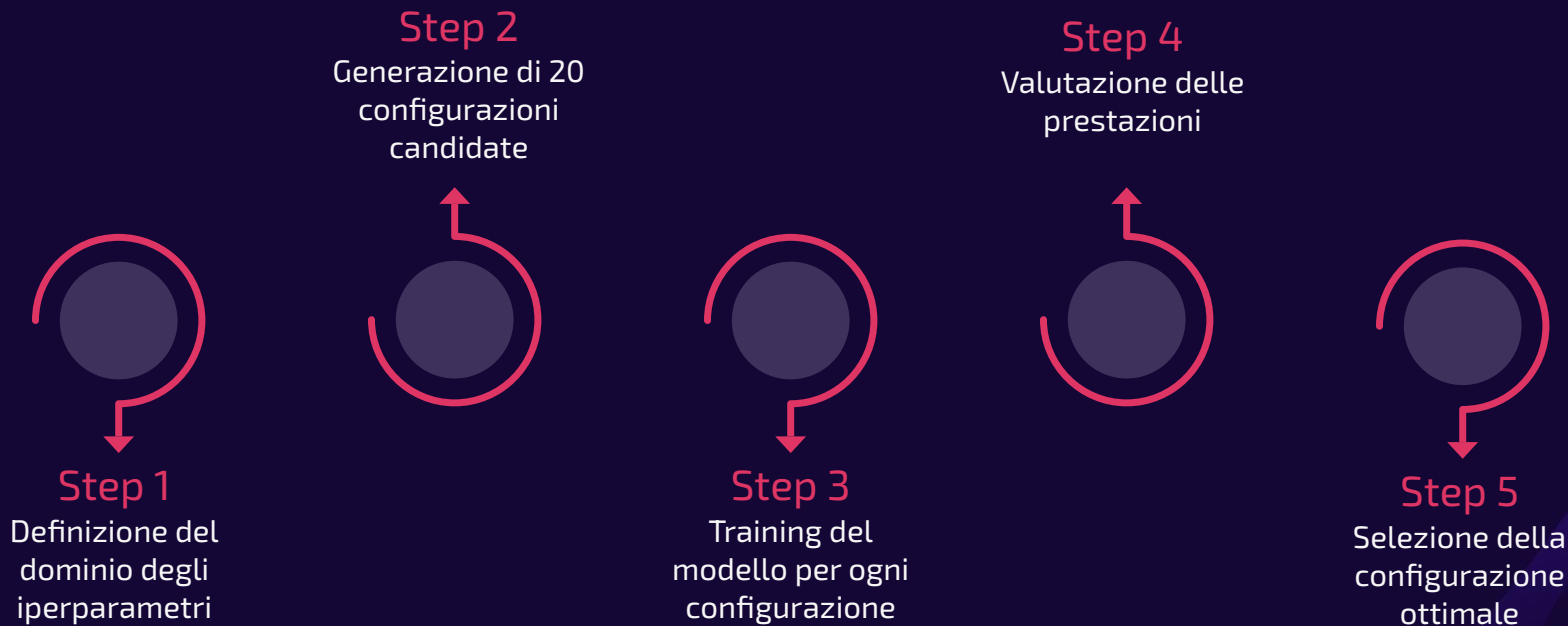
- ▶ Sostituisce Basis expansion con **multi-rate data sapling** (layer maxpool)
- ▶ Sottocampiona il segnale in ingresso a ritmi diversi, forzando i blocchi iniziali a concentrarsi sulle basse frequenze (macro-trend) e i blocchi finali sulle alte frequenze (dettagli giornalieri)
- ▶ **Hierarchical Interpolation:** La rete non prevede ogni singolo step temporale futuro, ma solo un set ridotto di punti, tramite interpolazione si ricostruire la serie completa



Riduce la complessità computazionale e i parametri del modello e previene l'overfitting permettendo di analizzare serie temporali e pattern più lunghi

Scelta dei parametri

La selezione degli iperparametri è stata effettuata tramite le rispettive versioni **Auto**, basate su **Optuna**.



Parametri individuati

Per rendere confrontabili le prestazioni dei diversi modelli, sono stati fissati lo stesso periodo di storico (14 giorni) e lo stesso orizzonte di previsione (14 giorni).

TCN

- context_size: 5
- learning_rate: $4e-3$
- max_steps: 1000
- batch_size: 32
- windows_batch_size: 1024
- scaler_type: 'identity'
- kernel_size: 3
- dilations: [1, 2, 4]

NBEATSx

- n_polynomials: 4
- n_harmonics: 8
- learning_rate: $2e-4$
- scaler_type: 'identity'
- max_steps: 500
- batch_size: 256
- windows_batch_size: 256
- step_size: 1

NHITS

- n_pool_kernel_size: [8, 4, 1]
- n_freq_downsample: [168, 24, 1]
- learning_rate: $5e-4$
- scaler_type: 'identity'
- max_steps: 1200
- batch_size: 256
- windows_batch_size: 256
- step_size: 1

Variabili esogene

Le variabili esogene sono informazioni aggiuntive rispetto alla serie temporale target che aiutano il modello a spiegare e prevedere l'andamento futuro dei giocatori.

Esogene storiche (Historical Exogenous Variables)

Informazioni osservate nel passato e utilizzate per identificare relazioni e pattern:

- ▶ **Reputazione:** Positive reviews, Negative reviews, Rating ratio
- ▶ **Popolarità:** Total reviews, Reviews_lag_7
- ▶ **Informazioni di calendario:** dow, is_weekend, month
- ▶ **Storico dei giocatori:** y_lag_1, y_lag_7, y_lag_14
- ▶ **Prezzo:** Price
- ▶ **Eventi:** Events

Esogene future (Future Exogenous Variables)

Informazioni note al momento della previsione:

- ▶ **Eventi futuri pianificati:** Events
- ▶ **Calendario futuro:** dow, is_weekend, month
- ▶ **Informazione storica disponibile:** y_lag_14

Multi Quantile Loss

La MQLoss è una funzione di perdita utilizzata per la valutazione di previsioni probabilistiche.

- ▶ Misura l'errore tra valori reali e previsioni su più quantili (es. 0.1, 0.5, 0.9)
- ▶ Penalizza in modo asimmetrico errori di sovrastima e sottostima
- ▶ Fornisce una misura più robusta dell'incertezza rispetto a una semplice loss puntuale

$$\text{MQL} (y_\tau, \{\hat{y}_\tau(q_1), \dots, \hat{y}_\tau(q_n)\}) = \frac{1}{n} \sum_{i=1}^n \text{QL} (y_\tau, \hat{y}_\tau(q_i))$$

Metriche di valutazione

- ▶ **RMSE (Root Mean Squared Error):** misura l'errore medio tra valori predetti e reali, penalizzando maggiormente gli errori più grandi
- ▶ **MAE (Mean Absolute Error):** media degli scarti assoluti tra predizioni e valori reali
- ▶ **sMAPE (Symmetric Mean Absolute Percentage Error):** errore percentuale simmetrico che misura la deviazione relativa tra predizioni e valori reali, bilanciando over e under-estimation
- ▶ **MDA (Mean Directional Accuracy):** misura la percentuale di volte in cui il modello prevede correttamente la direzione del cambiamento (crescita o diminuzione della serie)

$$\text{RMSE}(y_\tau, \hat{y}_\tau) = \sqrt{\frac{1}{H} \sum_{\tau=t+1}^{t+H} (y_\tau - \hat{y}_\tau)^2}$$

$$\text{MAE}(y_\tau, \hat{y}_\tau) = \frac{1}{H} \sum_{\tau=t+1}^{t+H} |y_\tau - \hat{y}_\tau|$$

$$\text{sMAPE} = \frac{100}{H} \sum_{\tau=t+1}^{t+H} \frac{|y_\tau - \hat{y}_\tau|}{\frac{|y_\tau| + |\hat{y}_\tau|}{2}}$$

$$\text{MDA} = \frac{1}{H-1} \sum_{t=1}^{H-1} 1(\text{sign}(y_{t+1} - y_t) = \text{sign}(\hat{y}_{t+1} - \hat{y}_t))$$

Risultati e Confronto delle prestazioni 1/5

TCN

RMSE

2535.55

MAE

2207.39

sMAPE

25.497

MDA

0.618

NBEATS_x

RMSE

1416.49

MAE

1246.74

sMAPE

14.102

MDA

0.727

NHITS

RMSE

1570.84

MAE

1387.76

sMAPE

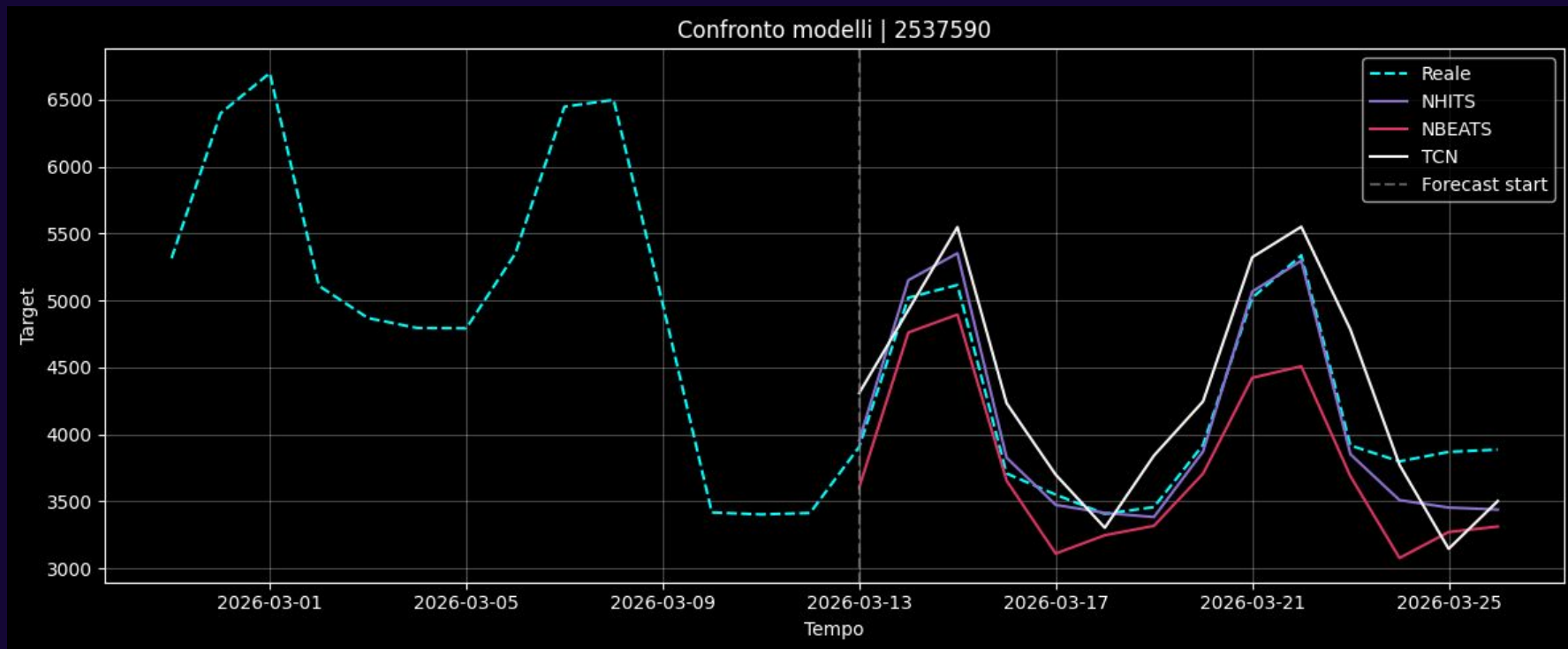
16.577

MDA

0.778

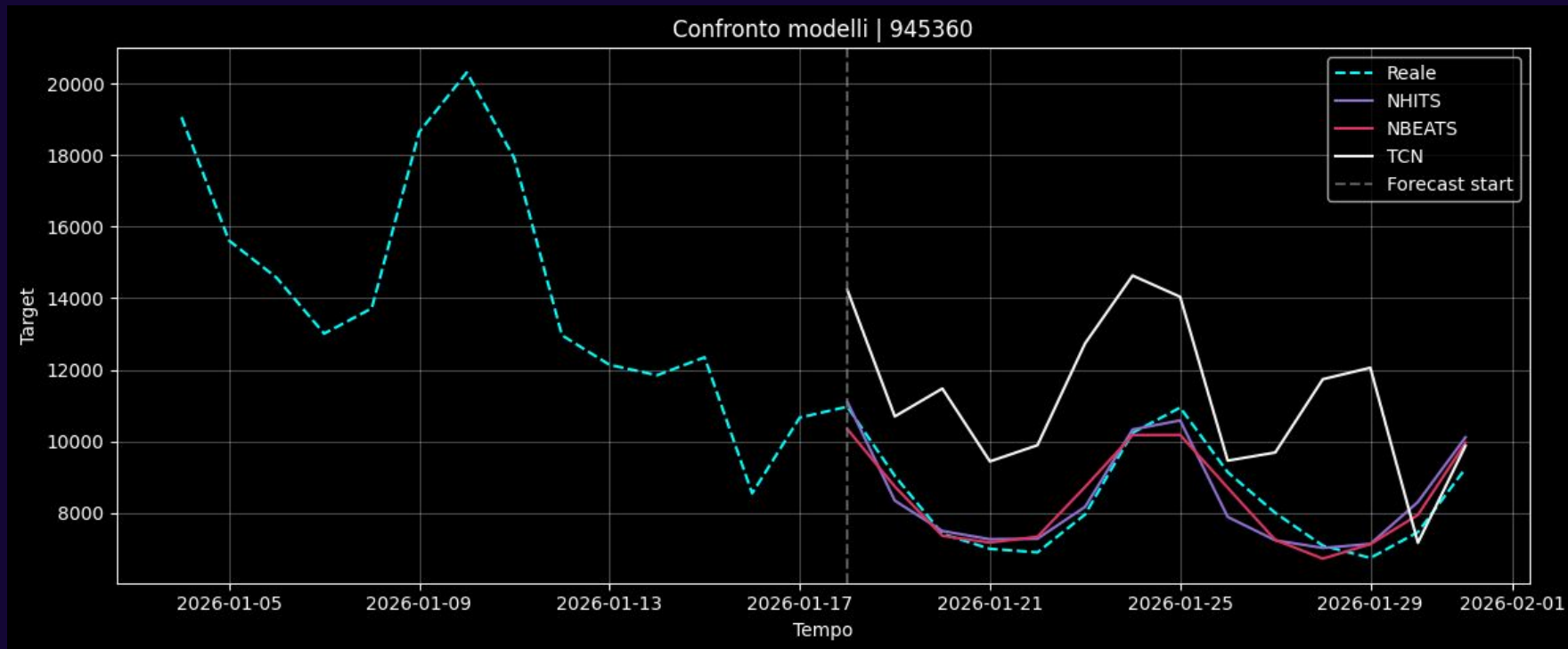
Risultati e Confronto delle prestazioni 2/5

Tutti i modelli riescono a catturare, con diversa accuratezza, l'andamento della serie.



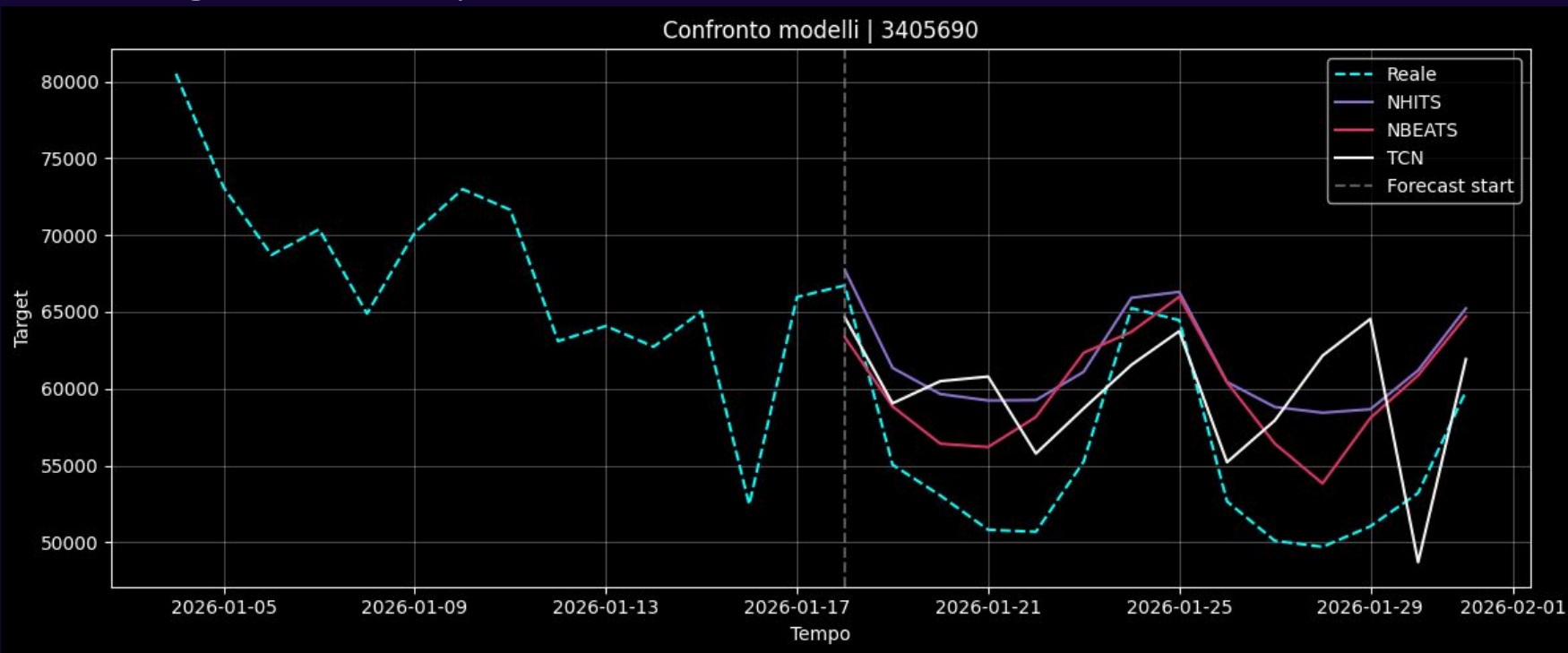
Risultati e Confronto delle prestazioni 3/5

In generale, NHITS e NBEATSx mostrano prestazioni migliori rispetto alla TCN.



Risultati e Confronto delle prestazioni 4/5

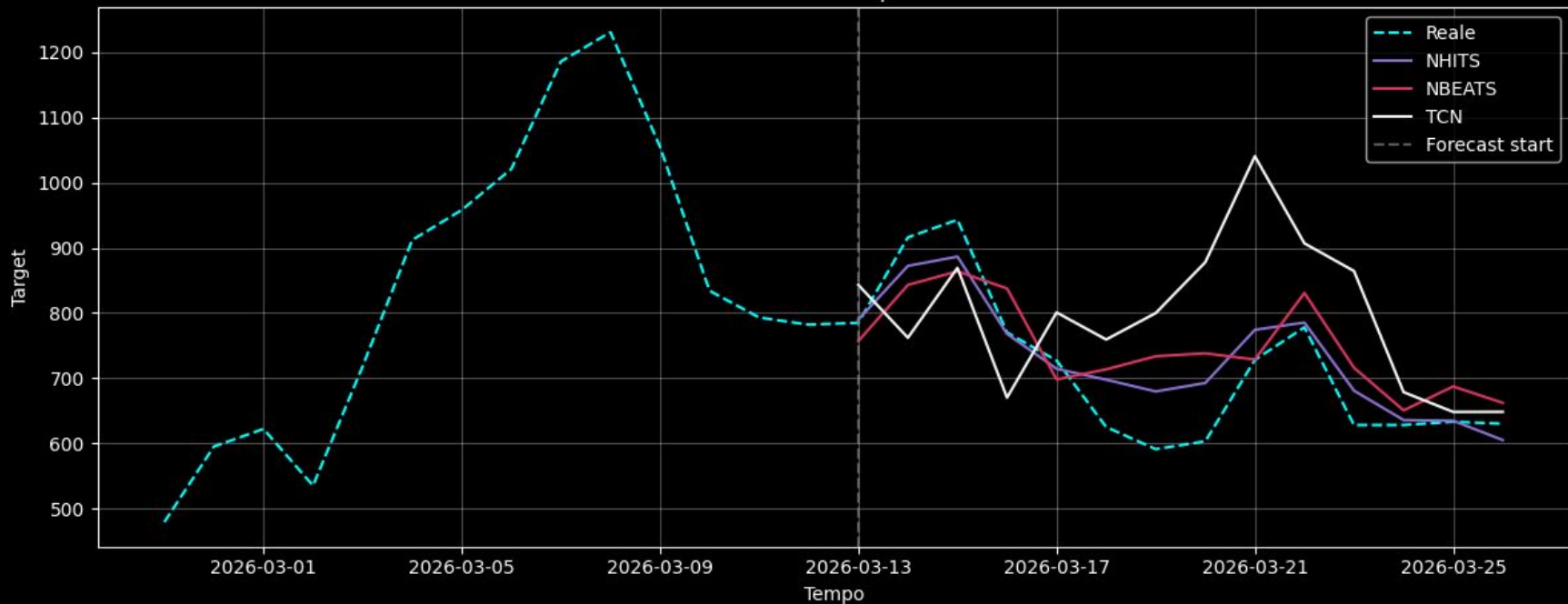
N-HITS genera una curva più fluida e stabile.



Risultati e Confronto delle prestazioni 5/5

N-HITS è più efficace nel catturare la direzione del trend.

Confronto modelli | 2124120



Conclusioni e lavoro futuro

Conclusioni

- ▶ Tutti i modelli analizzati sono in grado di catturare l'andamento generale del player count.
- ▶ N-BEATSx ottiene le migliori prestazioni in termini di RMSE, MAE e sMAPE. NHITS, grazie al sottocampionamento e all'interpolazione, produce previsioni più fluide e stabili, riducendo il rumore e catturando meglio la direzione delle variazioni del player count (MDA), a fronte di una lieve perdita di precisione.
- ▶ TCN ottiene invece prestazioni inferiori. Non avendo una scomposizione esplicita in trend e stagionalità (come la basis expansion di N-BEATS) o una struttura multi-frequenza (come N-HITS), la TCN fa molta più fatica a bilanciare contemporaneamente la stagionalità con le variabili esogene. Queste, infatti, vengono integrate in modo meno diretto e il loro effetto tende a essere diluito dai filtri convoluzionali, rendendo più difficile catturare eventi improvvisi e cambiamenti di trend.

Lavoro futuro

- ▶ Ampliamento del dataset, includendo un numero maggiore di giochi.
- ▶ Integrazione di nuove feature, come patch e festività.
- ▶ Estensione dell'orizzonte di previsione.



Grazie per la vostra
attenzione

Beatrice Ravaglia

Bibliografia

- ▶ NeuralForecast: <https://nixtlaverse.nixtla.io/neuralforecast>
- ▶ SteamDB: <https://steamdb.info/>
- ▶ TCN: <https://arxiv.org/abs/1611.05267>
- ▶ NBEATSx: <https://arxiv.org/abs/2104.05522>
- ▶ NHITS: <https://arxiv.org/abs/2201.12886>
- ▶ Time Series Forecasting (survey):
<https://link.springer.com/article/10.1007/s13042-025-02560-w>
- ▶ Time Series Forecasting (survey): <https://arxiv.org/abs/2411.05793>
- ▶ Evaluation Metrics:
<https://eshban9492.medium.com/understanding-evaluation-metrics-for-time-series-forecasting-5c8a3c877654>